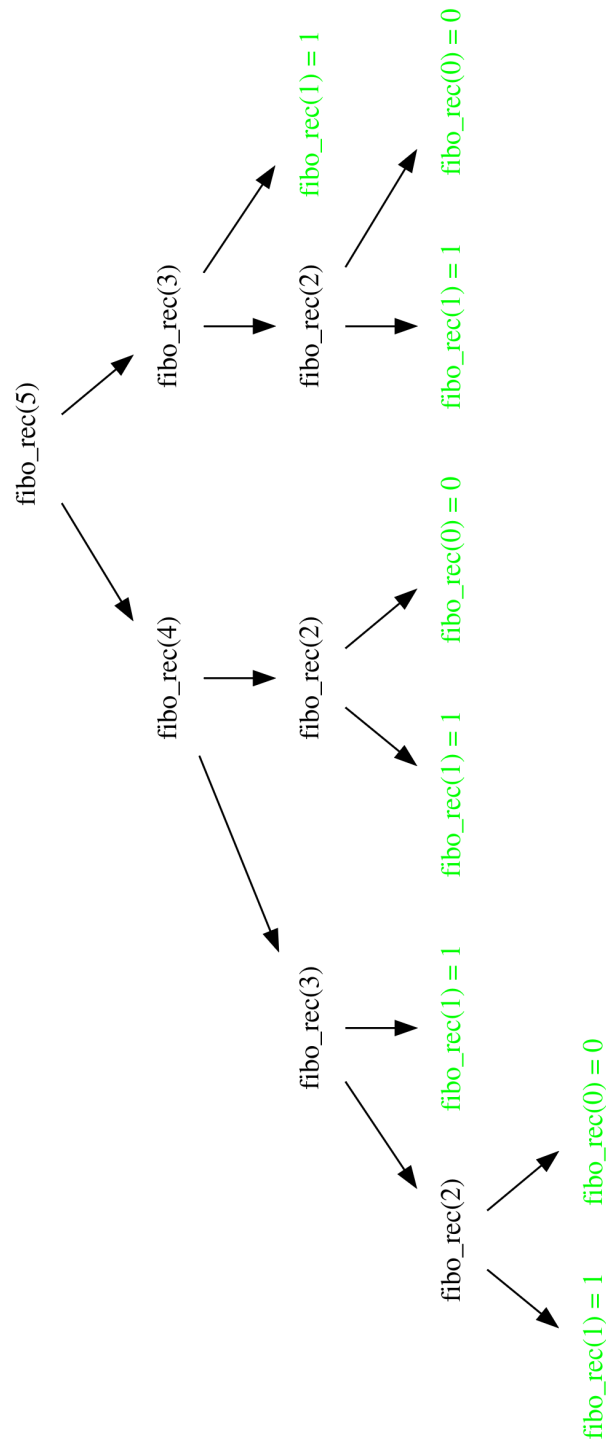


Corrigé – Programmation dynamique

1.

```
def fibo_rec(n):  
    if n <= 1 : # condition d'arrêt  
        return n  
    return fibo_rec(n-1) + fibo_rec(n-2)
```

2.



3. `fibo_rec(0)`, `fibo_rec(1)`, `fibo_rec(2)` et `fibo_rec(3)`

4. L'arbre est parcouru dans l'ordre d'un parcours en profondeur préfixe : d'abord le nœud courant puis les sous-arbres gauches, puis les sous-arbres droits.

5.

memo[0]	memo[1]	memo[2]	memo[3]	memo[4]	memo[5]
None	None	None	None	None	None
None	1	None	None	None	None
0	1	None	None	None	None
0	1	1	None	None	None
0	1	1	2	None	None
0	1	1	2	3	None
0	1	1	2	3	5

6. `memo = [None for _ in range(n+1)]`

```
7. def fibo_rec_memo(n, memo=[]):  
    if memo == []:  
        memo = [None for _ in range(n+1)]  
  
    if memo[n] is not None:  
        return memo[n]  
  
    if n <= 1:  
        memo[n] = n  
    else:  
        memo[n] = fibo_rec_memo(n-1, memo) + fibo_rec_memo(n-2, memo)  
  
    return memo[n]
```

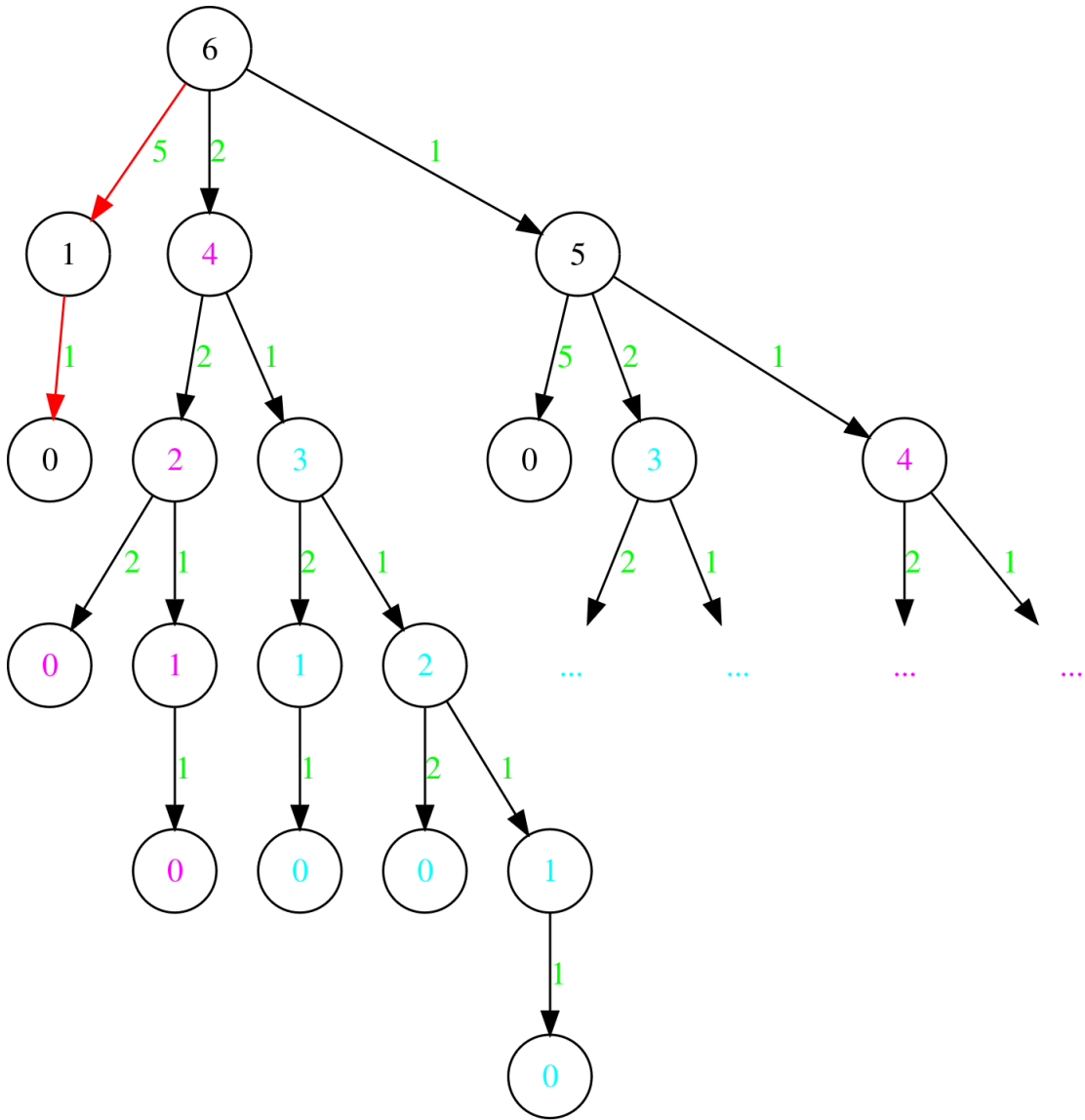
```
8. def fibo_ite_memo(n):  
    memo = [0, 1]  
  
    for i in range(2, n+1):  
        memo.append(memo[i-1] + memo[i-2])  
  
    return memo[n]
```

Travaux dirigés : le problème du rendu de monnaie

- 1. $20 + 20 + 5 + 2 + 1$, donc 5 pièces
- 2. $30 + 12 + 6$ (3 pièces). Ce n'est pas la solution optimale : on aurait pu rendre $24 + 24$ (2 pièces).
- 3.

	somme	nb_pieces	i	pieces[i]
init	48	0	0	30
1	18	1	0	30
2	18	1	1	24
3	18	1	2	12
4	6	2	2	12
5	6	2	3	6
6	0	3	3	6

4. (a)



- (b) Voir couleurs sur le dessin précédent.
- (c) En rouge sur le dessin précédent ($6 \rightarrow 1 \rightarrow 0$).

5.

```
def rendu_recuratif_dyn(somme, pieces, memo = []):  
    if memo == []:  
        memo = [None for _ in range(somme+1)]  
  
    if somme == 0:  
        return 0  
  
    if memo[somme] != None:  
        return memo[somme]  
  
    mini = somme  
    for p in pieces:  
        if p <= somme:  
            nb = 1 + rendu_recuratif_dyn(somme-p, pieces, memo)  
            if nb < mini:  
                mini = nb  
                memo[somme] = nb  
  
    return mini
```

6. Elle est beaucoup plus rapide à faire son travail et notre ordinateur tient le coup !

7.

```
def rendu_ite_dyn(somme, pieces):  
    memo = [0]  
    for s in range(1, somme+1):  
        memo.append(s)  
        for p in pieces:  
            if p <= s:  
                memo[s] = min(memo[s], 1 + memo[s-p])  
    return memo[somme]
```

8.

```
assert rendu_ite_dyn(48, euro) == 5  
assert rendu_ite_dyn(48, imperial) == 2  
assert rendu_ite_dyn(6, euro) == 2
```